

Reviewer Pack — Minimal Order of p-Adic Groups over Boolean Function Entropy

Entry 899798ef1440 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 06:44:24 UTC

Minimal Order of p-Adic Groups over Boolean Function Entropy

Entry ID: 899798ef1440 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 06:44:24 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): Complexity Theory: Boolean Function Entropy

Statement:

[‘For a given Boolean function f , the minimal order of its image under the continuous homomorphism from p-adic integers to complex numbers, denoted as $\text{ord}(\text{padic_order}(f))$, is upper bounded by the entropy $H(f)$ of the function.’, ‘Formally, for all Boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$, there exists a constant $c > 0$ such that $\text{ord}(\text{padic_order}(f)) \leq c * H(f)$.’, ‘Additionally, if two Boolean functions f and g have the same entropy but different minimal orders of their images under the homomorphism, then there exist polynomial-time computable functions h_f and h_g such that $\text{ord}(\text{padic_order}(h_f(f))) = \text{ord}(\text{padic_order}(h_g(g))) = \min(\text{ord}(\text{padic_order}(f)), \text{ord}(\text{padic_order}(g)))$.’]

Rationale (proposer’s reasoning):

[‘p-adic analysis provides a framework to study the continuity and convergence of functions in the presence of small errors. By relating the minimal order of p-adic groups to Boolean function entropy, this conjecture aims to uncover a deeper connection between these two fields.’, ‘The potential for a direct mapping between minimal orders and entropies suggests that the structure of Boolean functions might be reflected in the properties of p-adic numbers, potentially leading to new insights into complexity theory.’]

Taxonomy category: p_adic_analysis_boolean_entropy (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 97a510f39e6ef8c6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all tested Boolean functions f with entropy $H(f)$, the ratio $\text{ord}(\text{padic_order}(f)) / H(f)$ is less than or equal to a constant c , and no seed produces a ratio greater than c .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic analysis AND Boolean function entropy - minimal order p-adic groups AND entropy $H(f)$ - homomorphism p-adic integers to complex AND Boolean function Entropy

Top relevant hits considered: - [<http://arxiv.org/abs/math-ph/0512018v2>] On Phase Transitions for P -Adic Potts Model with Competing Interactions on a Cayley Tree - [<http://arxiv.org/abs/2201.08874v2>] Note on p -adic Local Functional Equation - [<http://arxiv.org/abs/2207.11806v3>] P -adic incomplete gamma functions and Artin-Hasse-type series - [<http://arxiv.org/abs/2503.11383v3>] Study of $\varphi \rightarrow K\bar{K}$ and $K_S^0 - K_L^0$ Asymmetry in the Amplitude Analysis of $D_{\{s\}^{\{+\}} \rightarrow K_{\{S\}^{\{0\}K_{\{L\}}}$ - [<http://arxiv.org/abs/2510.20330v1>] Precision Measurement of $D_s^{*+} - D_s^+$ Mass Difference with $D_{\{s\}^{\{*\}+} \rightarrow D_{\{s\}^{\{+\}(\rightarrow K_{\{+\}} K^{\{-} \pi^{\{+\}}) \pi}$ - [<http://arxiv.org/abs/2010.14527v3>] GWTC-2: Compact Binary Coalescences Observed by LIGO and Virgo During the First Half of the Third Observing Run - [<http://arxiv.org/abs/1309.3982v3>] Desingularization of complex multiple zeta-functions, fundamentals of p -adic multiple L -functions, and evaluation of - [<http://arxiv.org/abs/1508.07185v2>] Fundamentals of p -adic multiple L -functions and evaluation of their special values

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def entropy(f):
        n = len(list(f.keys()))
        counts = [sum(1 for _ in f.values() if v == i) / n for i in range(2)]
        return -sum(p * math.log2(p) for p in counts if p > 0)

    def padic_order(f):
        # Placeholder function to simulate the computation
        # Replace with actual implementation as needed
        return random.randint(1, 100)

    def map_functions(f, g):
```

```

    # Placeholder function to simulate the mapping
    # Replace with actual implementation as needed
    return f, g

n = random.choice([5, 10, 15, 20, 30, 40])
f = {i: random.randint(0, 1) for i in range(n)}
ent_f = entropy(f)

if ent_f == 0:
    return {
        "metric_name": "ord(padic_order(f)) / H(f)",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

ord_f = padic_order(f)
if ord_f is None:
    return {
        "metric_name": "ord(padic_order(f)) / H(f)",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

g = {i: random.randint(0, 1) for i in range(n)}
ent_g = entropy(g)
ord_g = padic_order(g)

if ent_f == ent_g:
    h_f, h_g = map_functions(f, g)
    ord_h_f = padic_order(h_f)
    ord_h_g = padic_order(h_g)
    if ord_h_f != ord_h_g or ord_h_f != min(ord_f, ord_g):
        return {
            "metric_name": "ord(padic_order(f)) / H(f)",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": f"Counterexample: h_f({f}) -> {h_f}, h_g({g}) -> {h_g}"
        }

if ord_f > ent_f * 100:
    return {
        "metric_name": "ord(padic_order(f)) / H(f)",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": f"Counterexample: ord({f}) = {ord_f}, H({f}) = {ent_f}"
    }

return {
    "metric_name": "ord(padic_order(f)) / H(f)",
    "metric_value": ord_f / ent_f,

```

```

        "instances_tested": 1,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = next(result["counterexample"] for result in results if result["conjecture_holds"] == False)
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b95d8481.py", line 99, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b95d8481.py", line 38, in run_trial
    ent_f = entropy(f)
            ^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b95d8481.py", line 23, in entropy
    counts = [sum(1 for _ in f.values() if v == i) / n for i in range(2)]
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b95d8481.py", line 23, in <genexpr>
    counts = [sum(1 for _ in f.values() if v == i) / n for i in range(2)]
               ^
NameError: name 'v' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means we cannot verify the conjecture’s support or falsification. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12658
2	preregistration	ollama_remote	glm4:latest	0	0	5562
3	novelty	ollama_remote	glm4:latest	0	0	4706
4	novelty	ollama_remote	glm4:latest	0	0	8823
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23720
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6833
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9041
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10901
9	judge	ollama_remote	glm4:latest	0	0	15969

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98212 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/899798ef1440.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/899798ef1440.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/899798ef1440.tar.gz (if generated)